Lab 2: teacher guide (git and GitHub)

Minute-by-minute script for the lab: what to do, which commands everyone types, what to say, and what to check before moving on. The overview and file list are in README.md.

Rhythm for every step: explain one idea, everyone types it, ask for a raised hand ("who has pushed?"), then move on. Do not go on while a few people are still on the previous step.

Before the lab

Day before, send students:

- Install git (git-scm.com; on Windows keep the default "Git Credential Manager"), Python 3.9 or newer, and make a GitHub account.
- Test: create any repo on GitHub, clone it, make a commit and push. This is where login problems show up, and it saves you 10 minutes in class.

You prepare:

- Two files to upload (Moodle or a link): `lab2.zip` (the starting repo in a `book-crawler/` folder, with `student_guide.pdf` next to it) and `quotes.py` (the one file added in the PR step). Tell students at the start to unzip only `lab2.zip` and not to touch `quotes.py` until step 7. (Or upload `quotes.py` only when you reach step 7.)
- Slides or images: (1) the flow picture (working folder, staged, committed, GitHub, with `add` / `commit` / `push` / `pull` as arrows), (2) a branch picture, (3) a screenshot of a PR with a green check.
- The exact text for each change on a slide: step 4 (the `PAGES` value), step 5 (the rating line), step 7 (`quotes.py`). If students type different things, nothing on the projector matches their screen.
- Your own demo repo, with a PR ready to show reviews (a comment on a line, the checks area).
- Run the whole lab yourself once on a clean folder, with a timer.

One-time identity (put it on the first slide, 1 min):

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Without it the first commit fails with "Please tell me who you are".

Timing at a glance

Min	Step	Core idea	Cut if late
0-10	1. Talk 1 and download	why versioning	shorten the talk
10-15	2. Init, first commit, connect, push	local vs remote	no

Min	Step	Core idea	Cut if late
15-30	3. Talk 2 and secrets hands-on	what stays out of git	CI mention
30-35	4. Change and commit	a commit is a snapshot with a diff	no
35-40	5. Branch, change, commit	a branch is a separate line of work	no
40-45	6. Merge into `main`, commit again	merge joins a branch to `main`	the extra commit
45-57	7. Big change, PR, tests, merge, pull	a PR is a merge with a safety step; a red check blocks shipping	the red-cross demo is yours only; student PRs (demo only)
57-60+	8. Extras	log, hashes, undo, tests	everything

The buffer is built in: steps 1 to 7 are meant to take about 50 minutes, with questions.

---

1. Talk 1 and download (0-10 min)

Students download the files, unzip `lab2.zip` and open a terminal in the `book-crawler` folder while you talk. Tell them to **leave `quotes.py` alone** until step 7: if it is in the repo for the first commit, the pull request will have nothing to show.

Say:

- Versioning: every commit is a saved snapshot you can go back to and compare. You do not lose code, and you can see who changed what and why.
- Teams: several people work on the same project without overwriting each other (branches).
- GitHub is a server that holds your repo (the *remote*), so there is a backup and others can see it. Git is the tool on your machine, GitHub is the website. They are not the same thing.
- Show the flow picture once. Keep coming back to it.

Show the crawler (2 min, no more): open `crawler.py`. It loops over 2 pages, parses titles and prices, and writes `books.jsonl`. Point at the empty `API_KEY` line and say "this is where the key to the server goes, we will come back to it". Do not run it yet.

2. Init, first commit, connect, push (10-15 min)

Order matters: **commit first, create the venv after**, so `.venv/` is not in the first commit.

```
git init
git add .
git commit -m "initial commit"
git branch -M main
```

Then on GitHub: **New repository**, give it a name. **Do not tick** "Add a README", "Add .gitignore" or a license, the repo must be empty. Copy the URL, then:

```
git remote add origin https://github.com/<user>/<repo>.git
git push -u origin main
```

Refresh the GitHub page: the files are there. A browser window may open the first time to log in.

Say:

- `init` makes a hidden `.git` folder where the history lives.
- `add` puts changes in the staging area, `commit` saves a snapshot with a message, `push` sends your commits to the remote. Point at the arrows in the picture.
- `origin` is just the nickname for the remote URL. `-u` remembers it so later `git push` is enough.

Start the venv now, so it runs during the next talk:

```
python -m venv .venv
# Windows (PowerShell): .venv\Scripts\Activate.ps1
# Windows (cmd):        .venv\Scripts\activate.bat
# Mac/Linux:            source .venv/bin/activate
pip install requests beautifulsoup4 python-dotenv
```

About 10 s plus 7 s.

Check: hand up when `git log --oneline` shows one commit and GitHub shows the files.

3. Talk 2 and secrets hands-on (15-30 min)

Talk and typing alternate. Questions are expected here. Each sub-step is "explain, type, check". The story: a secret ends up in the code, we move it to `.env`, git catches us out because `.env` is already tracked, and we fix it for good with `.gitignore`.

3a. The key goes in the code (2 min). Run `python crawler.py` once: it works with the empty key. Now type a fake key into the empty line of `crawler.py`:

```
API_KEY = "sk-demo-1234567890abcdef"
```

Run it again, it works. *Say:* it works, but the key is now written in the code, and the code is what we share on GitHub. Anyone who can see the repo sees the key. Secrets do not live in code.

3b. Move it to `.env`, and git bites (3 min). Put the key in `.env` as a key-value pair:

```
API_KEY=sk-demo-1234567890abcdef
```

Say: `.env` is a local file for settings that differ per machine or are secret, one `NAME=value` per line. Git can be told to leave it out of the repo, so the code can use the key without publishing it.

Then run `git status`. `.env` is listed as **modified**. *Say (this is the trap, on purpose):* we created `.env` empty and committed it with the first commit, so git is tracking it. If we commit now, the key goes to GitHub with it. We catch it now, before it happens. Stop tracking it, but keep the file on disk:

```
git rm --cached .env
```

`git status` now lists `.env` under "deleted" (git stops following it) and also as untracked (the file still exists). *Say:* if the key had been committed and pushed, it would be in the history for good (`git log -p` would show it) and the only fix is to **rotate** it: make a new one and disable the old one. Public repos are scanned by bots within minutes.

3c. Load it from `.env` (3 min). In `crawler.py` add `import os` and `from dotenv import load_dotenv` at the top, call `load_dotenv()` before the key line, and replace the hardcoded key with:

```
load_dotenv()
API_KEY = os.getenv("API_KEY")
```

Run it again, it must still work. *Say:* `load_dotenv()` reads `.env` when the program starts and `os.getenv` picks up the value. Nothing secret stays in the code.

3d. `.gitignore` (2 min). Run `git status` again. `.env` is untracked now, and so are `.venv/` and `books.jsonl`. *Say:* `git add .` would take all of them, including `.env`: untracking a file does not stop git from adding it again. So how do we tell git to never look at these? We create `.gitignore`:

```
.env
.venv/
__pycache__/
books.jsonl
```

Say: it lists what git should leave alone: secrets (`.env`), the environment (big, rebuildable), generated output, and Python's cache. Run `git status`: they are gone from the list. The reverse order does not work: `.gitignore` alone would not have stopped tracking `.env`, that is why we needed `git rm --cached` first.

A note on `.venv/`: since Python 3.13 the venv folder contains its own `.gitignore` with `*`, so git skips it even without your line. On Python 3.12 or older it shows up as untracked and `git add .` would add about 1,200 files (19 MB). Either way, list it. If someone did commit it: `git rm -r --cached .venv` (`-r` because it is a folder).

Say (one sentence, no exercise): since `.env` stays out of git, people who clone a shared repo do not get it, so projects often add a `.env.example` with fake values to show which settings are needed. It is an extra step in the student guide.

3e. `requirements.txt` (2 min). Write it by hand:

```
requests
beautifulsoup4
python-dotenv
```

Then run `pip freeze` to show the full list (9 packages, with versions). *Say:* the hand-written file says what you chose, and `pip freeze` says exactly what is installed. Pinned versions (`requests==2.x.x`) make a rerun reproducible, unpinned are more flexible. For this lab, hand-written is fine.

Now the payoff. In step 2 we installed the three libraries one by one. With the file, one command installs all of them:

```
pip install -r requirements.txt
```

Everyone runs it. Each line says `Requirement already satisfied`, because the libraries are already there. *Say:* on a new computer, in a new venv, or for a teammate, this one command installs the whole setup, so nobody types the libraries one by one or has to guess which ones. It is also exactly what GitHub will do for us later in the pull request check.

Say (why we need both the venv and the file): the venv gives this project its own clean set of libraries, so projects do not break each other and nothing is installed on your whole machine. `requirements.txt` is the written list of what the project needs, so the venv can be rebuilt on any other computer with one command. The venv itself stays out of git (it is in `.gitignore`), and the list travels with the code.

3f. Commit, but do not push yet (2 min).

```
git status
git add .
git status
git commit -m "load key from .env, add .gitignore and requirements"
```

Make students run `git status` before committing. It is the habit that catches mistakes. Check that `.env` is **not** in the list of files to be committed, apart from being deleted from tracking. Say: "we push once, after the next change", so the push is shown one time, in step 4.

Mention only (1 min, if there is time): tests and CI. One slide: tests check that the code still works, and GitHub can run them automatically on every push (a green tick or a red cross). That is Lab 6. Point at the files that are already in their repo (`test_quotes.py` and the `.github` folder): "these will run for the first time in the pull request, not on a push".

Check: `git ls-files` lists `.gitignore`, `README.md`, `crawler.py` and `requirements.txt`, and no `.env`.

4. Change and commit (30-35 min)

On the slide: change `PAGES = 2` to `PAGES = 3` in `crawler.py`.

```
git diff
git add crawler.py
git commit -m "crawl 3 pages"
git push
git log --oneline
```

Say:

- `git diff` shows exactly what changed, line by line: a one-line change gives a one-line diff. This is what a reviewer will see later.
- A commit message says *why*, so future-you can find it.
- `git log --oneline` shows a chain: each commit points to the one before it. The short code on the left is its ID (more on that at the end).
- **This one `git push` sends both commits** (the secrets fix from step 3 and this change). That is why step 3 stopped at the commit: the push is shown once. Two commits, one push.

Check: `git log --oneline` shows 3 commits, and on GitHub there is no `.env` and no key anywhere in the files.

If someone's `git commit` opens a strange editor, they forgot `-m`. Press `Esc`, type `:q!` and Enter, then retry.

5. New branch, change, commit (35-40 min)

```
git branch new-feature
git switch new-feature
```

(`git branch` alone lists branches, `*` marks the current one. `git switch -c new-feature` does both at once, mention it.)

On the slide, change the line in `crawler.py` that appends a book to also store the rating:

```
        books.append({"title": item.h3.a["title"], "price":
item.select_one(".price_color").text, "rating": item.select_one(".star-rating")
["class"][1]})
```

Run the crawler and open `books.jsonl` to see the new field, then:

```
git add crawler.py
git commit -m "add rating"
git push -u origin new-feature
```

Say:

- A branch is a separate line of work. `main` stays as it was, so you can try things without risk. Show the branch picture.
- Branches are cheap in git: they are just a pointer to a commit.
- On GitHub, use the branch dropdown to show the new branch next to `main`.

Check: `git branch` shows `* new-feature` and GitHub lists both branches.

6. Merge into main, then commit again (40-45 min)

```
git switch main
```

Show before merging: open `crawler.py`. The rating is **gone**. Switch back and forth once if you want to make the point that the files on disk change when you switch branches.

```
git merge new-feature
git log --oneline
git push
```

The output says `Fast-forward`.

Say (correct the word): this is a **merge**, not a pull. Merge is how the work of a branch joins another branch. `main` had no new commits since the branch started, so git only moves its pointer forward (fast-forward), and the two now match. Pull is different: it downloads from GitHub, and we meet it in step 7.

Optional extra commit on `main` (2 min, cut if late): add a line to `README.md`, such as "Output: books.jsonl", and commit and push. It moves `main` forward before the next branch.

Delete the finished branch if you want to show it: `git branch -d new-feature`.

7. Big change, pull request, tests, merge, pull (45-57 min)

Students copy one file, `quotes.py`, into their repo folder (they downloaded it with the other files and have not opened it). It crawls `quotes.toscrape.com` and writes `quotes.jsonl`, and it has a small function `clean()`.

The rest is already in their repo, from the first commit: `test_quotes.py` (one test, one `assert`, that checks `clean()`) and `.github/workflows/test.yml` (tells GitHub to run the tests on every **pull request**). The workflow does not run on a plain push, so nothing ran until now. Until `quotes.py` exists, `pytest` fails with an import error, so tell students **not to run `pytest` before this step**.

```
git branch quotes
git switch quotes
# copy quotes.py into the repo folder
python quotes.py
git status
git add quotes.py
```

```
git commit -m "add quotes crawler"
git push -u origin quotes
```

Add the file by name (not `git add .`): running `quotes.py` created `quotes.jsonl`, which should not be committed. `git status` shows it as untracked, a good moment to say so. (You can also add `quotes.jsonl` to `.gitignore`.) The pull request then shows exactly one new file.

On GitHub:

1. A yellow banner "quotes had recent pushes: **Compare & pull request**". Click it.
2. Write a title and a short description. *Say:* the description is for the reviewer: what, why, how to test.
3. Open **Files changed**. This is the diff of the branch against `main`. The reviewer reads this.
4. **The check.** On the PR page, the checks area at the bottom shows a yellow dot ("tests" running) and, after 20-40 seconds, a **green tick**. Click **Details** to open the log once: GitHub started a clean machine, installed the libraries from `requirements.txt` and ran `pytest`. *Say:* the test ran on a computer that is not yours. It had nothing but your repo, so `requirements.txt` must list what the project needs. If a student left a library out of it, their check fails with `ModuleNotFoundError`: a real example of why the file matters.
5. **Demo on your repo (yours, not theirs): the red cross.** Change the expected value in `test_quotes.py` (for example `"Hello"` to `"Hello!"`), commit and push to the same branch. The same PR re-runs the check and it turns **red**. Click **Details** to read the error. *Say:* in a real team a red check means this cannot be merged until someone fixes it. Teams enforce it with a repo setting called branch protection that disables the merge button while checks fail (show it on a slide or in your repo's settings, do not set it up live). Undo the change, push again, the check goes green. Optionally show the "This branch has conflicts" banner on a PR prepared in advance.
6. Everyone clicks **Merge pull request** and **Confirm**. Optionally **Delete branch**.

Say: a PR is a proposal to merge a branch. It is the same merge as in step 6, with a place to review and test before it happens. In a team nobody pushes straight to `main`, and a failing check stops a change from shipping. We build this properly in Lab 6.

If a student's check does not start (new accounts occasionally have Actions blocked), they follow along on your screen and merge anyway.

Now `main` on GitHub is ahead of your laptop. Back in the terminal:

```
git switch main
git status
git pull
git log --oneline --graph
```

`quotes.py` appears in the folder, and the graph shows the branch splitting off and rejoining (GitHub's merge button always makes a merge commit).

Say: **pull downloads what is new on GitHub and merges it into your branch.** It is the opposite arrow of push. If you only want to see what is new without changing anything, `git fetch`.

Check: the PR showed a green tick, `git log --oneline --graph` shows a merge commit, and `quotes.py` exists on `main`.

8. Extras if there is time (57-60+ min)

Pick what the group asks about. In rough order of value:

- **Log options:** `git log --oneline --graph --all`, `git log -p` (with diffs), `git log -3`, `git show <id>` (one commit).
- **Hashes:** each commit has an ID, a long hexadecimal number computed from its content, its message, its author and the previous commit. Change anything and the ID changes. `--oneline` shows the first 7 characters, and any unique start works in commands: `git show a1b2c3d`.
- **Undo, the three honest answers:**
  - Changed a file, not committed, want it back: `git restore <file>`.
  - Committed, not pushed, want to unstage: `git reset --soft HEAD~1` (explain only if asked).
  - Already pushed: `git revert <id>` makes a new commit that undoes it. Do not rewrite shared history.
- **`.git` folder:** `ls .git`. It is the whole history. Delete it and the repo is gone, but the files stay.
- **Conflicts (demo on your screen only):** two branches change the same line, merge the second, show the `<<<<<<<` markers. When two people change the same line, git asks a human to choose.
- **Tests (they have seen one in the PR):** show `test_quotes.py` again line by line: a function name that starts with `test`, one `assert`. Run `pytest` locally (`pip install pytest` first). Mention "one test per function" as the habit, and that the workflow file is what runs it automatically. It leads to Lab 6.

Common problems

Symptom	Cause	Fix
"Please tell me who you are" at the first commit	no identity set	the two `git config --global` lines
`git push` rejected, "unrelated histories" or "fetch first"	GitHub repo was created with a README or `.gitignore`	easiest: delete the GitHub repo and create an empty one, then `git push -u origin main`
"src refspec main does not match any"	branch is called `master`, or no commit yet	`git branch -M main` after at least one commit
Push asks for a password and fails	GitHub no longer accepts passwords	log in through the browser prompt (Git Credential Manager), or use a personal access token; if it drags on, pair the student with a neighbour
`.venv` or `books.jsonl` was committed	`git add .` before `.gitignore`	`git rm -r --cached .venv`, `git rm --cached books.jsonl`, then commit
PowerShell: "running scripts is disabled" when	execution policy	`Set-ExecutionPolicy -Scope Process Bypass`, or use cmd

Symptom	Cause	Fix
activating the venv		
`pip` installs into the wrong Python	venv not active	`python -m pip install ...`; the prompt should show `(.venv)`
`LF will be replaced by CRLF` warning	Windows line endings	harmless, ignore
`git commit` opens a text editor	no `-m`	`Esc`, `:q!`, retry with `-m "..."`
Merge says "Already up to date"	merging in the wrong direction or branch has no commits	check `git branch`, you must be on `main` and the branch must have a commit

Questions students ask

- *Can I delete a key from the history?* Technically yes, by rewriting history, but anyone who cloned already has it. Rotate it, then clean up if needed.
- *Why not just work on `main`?* Alone, you can. In a team, a broken change on `main` stops everyone. Branches and PRs keep `main` working.
- *What is the difference between git and GitHub?* Git is the version control tool on your computer. GitHub is a website that hosts git repos and adds PRs, issues, CI.
- *Do I commit often?* Yes, small commits with clear messages. It is easy to combine small ones and hard to split a big one.